1. Text Preprocessing

본 프로젝트는 Rotten Tomatoes 영화 리뷰 데이터셋을 활용하여 감성 분류를 수행하였으며, 모델 학습에 앞서 텍스트 데이터의 품질을 높이기 위한 전처리 과정을 설계하고 반복적으로 개선하였습니다.

1.1 초기 전처리 계획

초기에는 일반적인 NLP 전처리 흐름을 따랐습니다.

- 특수문자 제거: re.sub(r"[^a-zA-Z0-9]", " ", text)로 알파벳과 숫자 외 문자를 제거함.
- 소문자 변환 및 토큰화: word_tokenize(text.lower())로 처리.
- stopwords 제거: NLTK의 영어 stopwords 리스트를 사용.
- Lemmatization: WordNetLemmatizer 사용.

1.2 최종 전처리 방식 및 변경 이유

여러 실험 결과, 구두점을 제거하면 성능이 오히려 하락한다는 점을 확인하였고, 최종적으로 다음과 같은 방식으로 고정했습니다:

- 구두점 제거 유지: nltk의 word_tokenize로 토큰화 후, 특수문자를 제거한 상태를 유지.
- Stopwords 제거하지 않음: 감성 분석에서는 "not good"과 같이 부정어가 중요한 역할을 하므로 stopwords는 제거하지 않았습니다.
- 불필요한 단어 제거: 의미 없는 토큰들(rrb, lrb, n, u)만 제거.
- Lemmatization 유지: 단어 형태를 일반화하여 모델이 학습할 단어 수를 줄이고자 함.
- 시퀀스 길이 설정: 문장 길이의 평균과 표준편차를 기준으로 max_len = mean + 2*std를 설정하였고, 이 길이에 맞춰 padding 또는 truncation 수행.

2. A model Description

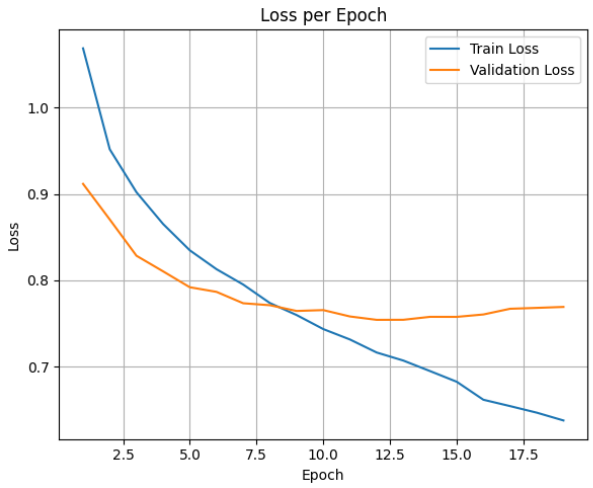
본 과제에서는 사전학습 모델 사용이 금지되고 PyTorch만 허용되므로, 직접 구현한 RNN 기반 모델을 사용했습니다. 최종 모델은 다음과 같은 구조입니다:

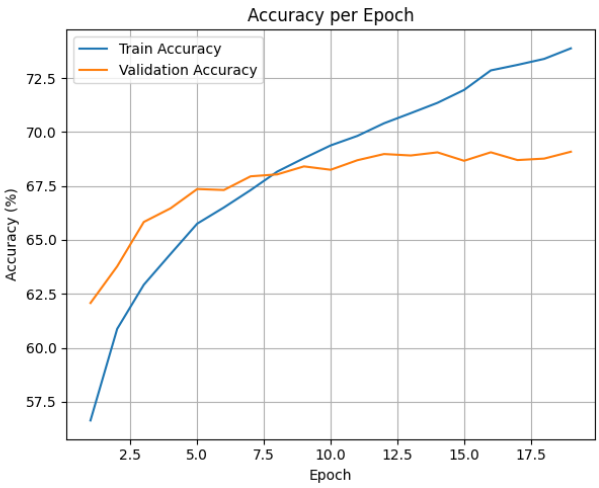
2.1 임베딩 레이어

- GloVe (glove.6B.100d.txt)를 활용한 사전학습 임베딩 사용.
- freeze=False로 설정하여 임베딩 레이어도 학습 과정 중 fine-tuning 가능하도록 함.

2.2 RNN 구조

- LSTM (hidden_dim=256, 128) → GRU (64, 32) 의 4계층 구조.
- 모든 RNN 레이어는 bidirectional=True로 설정하여 양방향 문맥 정보를 반영.
- 각 RNN 레이어 뒤에는 Dropout이 적용되어 과적합을 방지.
- 마지막 GRU의 양방향 hidden state를 concat하여 fully connected layer로 전달.
- 최종 출력은 5-class classification을 위한 Softmax 레이어.





3. Scheme changes for hyperparameters

3.1 Embedding Layer Fine-Tuning

- Initial approach: GloVe pretrained embedding을 freeze=True로 고정하여 사용했습니다. 이는 학습 속도를 빠르게 하고 overfitting을 방지하려는 목적이었습니다.

- Final decision: 최종적으로는 freeze=False로 설정하여 embedding layer도 학습 대상에 포함되도록 했습니다. 이는 Rotten Tomatoes 데이터셋의 특수한 감성 표현과 문맥을 반영하기 위해 embedding을 fine-tuning할 수 있게 하여 모델의 표현력을 향상시키는 데 큰 기여를 했습니다.

→ 결과적으로 Kaggle score는 0.64033 → 0.66487로 상승하였습니다.

3.2 Optimization & Regularization

- **Optimizer:** 초기에는 일반적인 Adam optimizer를 사용했으나, weight decay의 적용을 보다 정확히 반영하기 위해 AdamW로 교체하였습니다.

- **Learning Rate**: 여러 실험을 통해 0.001, 0.0001 등 다양한 값을 테스트한 결과, 5e-4 가 가장 안정적이면서 빠른 수렴 속도를 보였습니다.

- **Weight Decay**: overfitting을 방지하고 모델 복잡도를 제어하기 위해 1e-5 값을 설정하였습니다.

- **Dropout:** dropout은 0.5 비율로 embedding 뒤 (self.dropout_embedding)와 RNN 출력 뒤 (self.dropout_rnn_output)에 각각 적용되었으며, 이는 과적합 방지에 효과적이었습니다.

추가적으로 0.3 / 0.7 등 다양한 비율도 실험하였으나, 0.5가 가장 일관되고 안정적인 성능을 보여 최종 설정으로 확정하였습니다.

- **Loss Function**: CrossEntropyLoss를 사용했습니다.

3.3 Scheduler & Early Stopping

- **Learning Rate Scheduler:** 검증 손실(val_loss)이 일정 에폭 동안 개선되지 않을 경우 학습률을 자동으로 감소시키는 ReduceLROnPlateau scheduler를 도입하였습니다.

mode='min', factor=0.5, patience=2 설정을 사용하여 수렴이 정체될 때 더 세밀한 최적화를 유도할 수 있도록 했습니다.

- **Early Stopping:** validation loss 기준으로 patience=7 설정을 적용하여, 성능 향상이 멈출 경우 학습을 조기에 종료하였습니다. 이 전략은 불필요한 연산 자원을 절약하고 overfitting을 방지하는 데 효과적이었습니다.

3.4 Batch Size & Training Strategy

- **Batch Size:** 실험 초기에는 64로 설정하였으나, 메모리 효율성과 수렴 안정성을 고려하여 최종적으로 128 로 확정하였습니다.

- **Train/Validation Split:** 라벨 불균형 문제를 고려하여 StratifiedShuffleSplit을 사용하여 데이터셋을 분리함으로써, 각 class의 비율이 train/val에 고르게 분포되도록 했습니다.

3.5 추가사항

- Self-Attention 적용

RNN 위에 간단한 Self-Attention 구조를 추가하여 문장의 중요한 부분에 더 집중할

수 있도록 시도했습니다. 하지만 validation 정확도에서 일관되게 하락이 발생했고, 오히려 과적합이 유발되는 경향을 보여 최종 모델에서는 제외하였습니다.

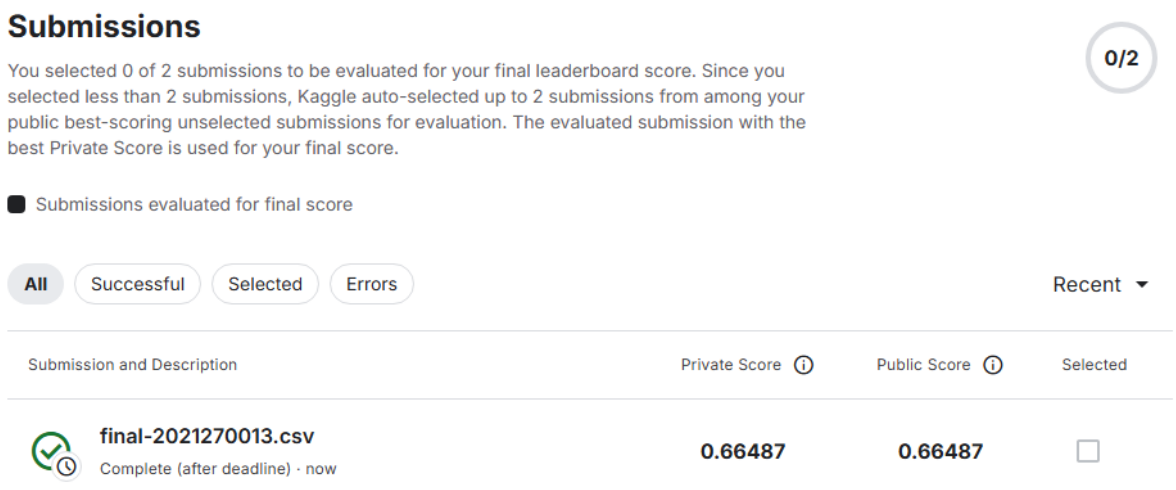
- **Dropout 분리 적용**

embedding dropout과 RNN output dropout을 별도로 분리하여 각각 다른 값으로 설정하는 실험을 진행했습니다. 예를 들어, 임베딩에는 0.3, RNN 출력에는 0.5를 적용하는 식이었지만, 결과적으로 성능 개선 효과가 없었고, 학습 안정성에도 특별한 이점이 없어 다시 단일 dropout 설정으로 복귀하였습니다.

- **Transformer 구조 실험**

PyTorch의 nn.TransformerEncoderLayer를 활용한 간단한 Encoder-only Transformer 모델도 구현해보았습니다. 그러나 학습 과정에서 기울기 폭주(gradient explosion)가 반복적으로 발생했고, gradient clipping 및 LayerNorm을 도입해도 해결되지 않아 학습 안정성이 매우 떨어졌습니다.
데이터셋의 특성과 모델의 복잡도를 고려했을 때, Transformer보다는 RNN 기반 구조가 더 안정적으로 작동한다고 판단하여 Transformer 구조는 채택하지 않았습니다.

4. Evaluation



- Kaggle Public Score: 0.66487

이는 초기 모델(0.64033) 대비 의미 있는 성능 향상을 보여줍니다. 특히 embedding fine-tuning, 하이브리드 LSTM-GRU 구조, dropout 및 scheduler 조합이 최종 성능에 긍정적인 영향을 미친 것으로 분석됩니다.